BugScout AI: Agent-Based LLM System for Real-Time Issue Detection and Resolution

Team Name: KARLVerse

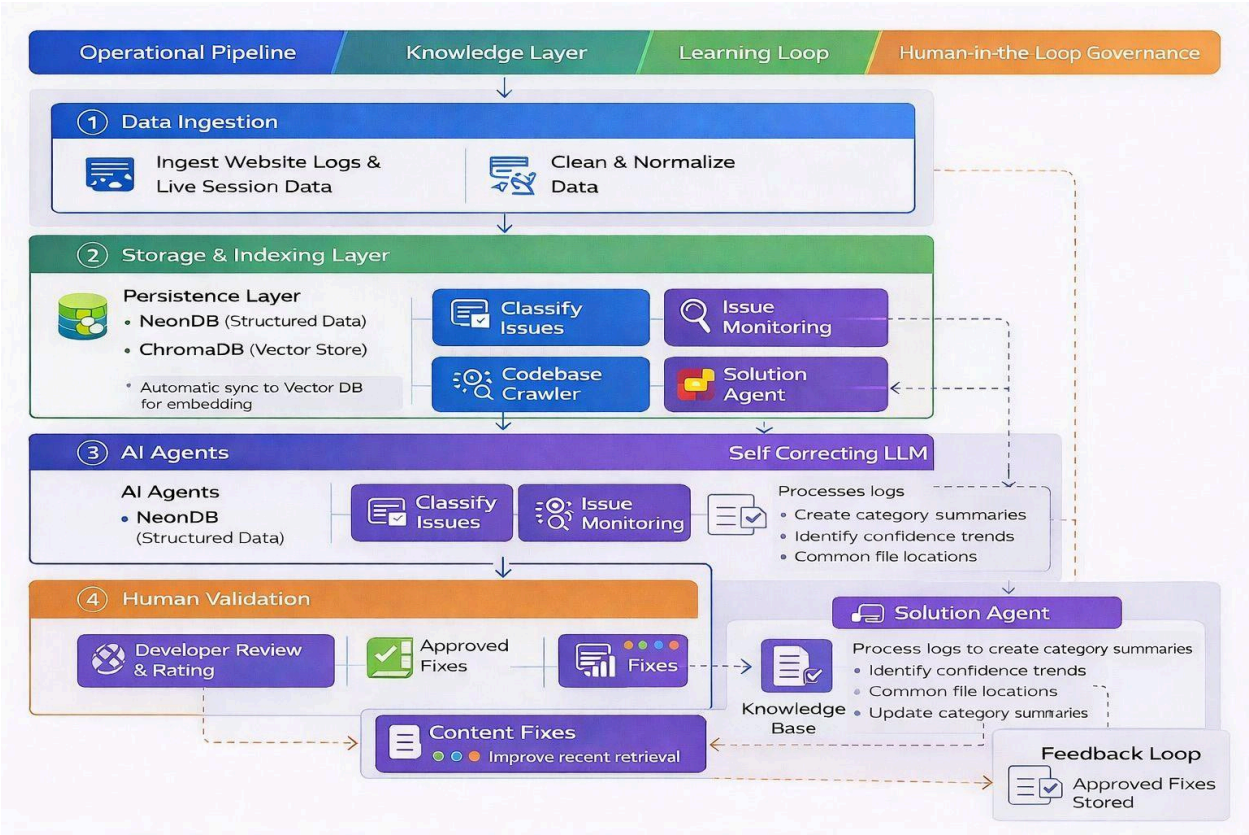
Team Members:
Krishay Nair
Rimaansh Bhargava
Kanishk Kapoor
Vanshika Wadhwa

Project Title: BugScout AI

Project Summary: BugScout AI is a self-learning, multi-agent LLM system that detects real-time UX friction in production, maps it to your codebase, and delivers governed, continuously improving fixes under human oversight.

System Overview

BugScout AI is a self-learning production monitoring system that automatically detects, diagnoses, and suggests code-level fixes for web application issues by analyzing real-time PostHog session replays. The system employs a coordinated four-agent LLM architecture: Issue Monitoring Agent, Codebase Crawler Agent, Solution Agent, and Self-Learning Agent, that work together to transform raw user session data (exceptions, rage clicks, dead clicks, console errors) into precise, actionable fixes with correct file paths and code snippets. Unlike generic LLMs, BugScout AI learns continuously from developer feedback, building category-specific knowledge summaries that enables improvement in code location accuracy. The entire pipeline operates on private production data from multiple domains (academic platforms, industrial software, enterprise SaaS, telecommunications), stored in NeonDB for integrity and ChromaDB for semantic retrieval, creating a unique dataset that baseline models cannot access.



Pipeline

1. Real-Time Data Ingestion

BugScout AI starts by collecting website logs, live session traces, API errors, and system events in real time. Data is collected in two primary ways:

- Live production data from partner company websites (real user interactions, not synthetic test data)
- Private institutional datasets securely provided by companies across multiple domains, including academic platforms, industrial automation systems, enterprise SaaS products, and telecommunications infrastructure

This gives BugScout AI access to multi-domain, real-world production environments, making the dataset highly novel and difficult to replicate.

This level of data novelty and access difficulty creates a competitive advantage, as baseline models and public systems cannot replicate or access similar private, production-grade session streams.

2. Data Cleaning & Normalization

Raw logs often contain noise, duplicates, and inconsistent formats. The preprocessing layer cleans, deduplicates, and normalizes this data.

This step ensures that anomaly detection, classification, and summarization operate on high-quality inputs, significantly improving model performance and reducing false positives.

3. Storage & Intelligent Indexing Layer

BugScout AI uses a dual storage architecture:

- Structured Database (NeonDB) for categorized logs and metadata
- Vector Database (ChromaDB) for semantic embeddings and contextual search

Logs are automatically embedded and indexed, enabling both keyword-based retrieval and semantic reasoning. This powers retrieval-augmented generation (RAG) for accurate solution generation.

4. Multi-Agent AI Orchestration

BugScout AI operates through specialized AI agents, each responsible for a specific task:

- Issue Classification Agent: Categorizes errors into predefined or dynamically generated issue types
- Issue Monitoring Agent: Detects recurring patterns, anomaly spikes, and confidence trends
- Codebase Crawler Agent: Maps detected issues to relevant files and code dependencies
- Solution Agent: Generates context-aware fixes using retrieved documentation, past fixes, and embeddings

This modular agent-based architecture enables scalability, explainability, and parallel processing.

5. Self-Correcting LLM Engine (Core Intelligence Layer)

At the heart of BugScout AI lies the Self-Correcting LLM.
Unlike static models, this LLM:

- Processes logs to create dynamic category summaries
- Identifies confidence trends in predictions
- Cross-verifies generated solutions against retrieved context
- Refines outputs based on historical approvals

The model continuously evaluates its own responses and improves through structured feedback, reducing hallucinations and increasing precision over time.

6. Human-in-the-Loop Validation

Proposed fixes generated by the Solution Agent are sent to developers for structured review. They can approve, rate, modify, or reject AI-generated outputs before deployment.

This ensures full human oversight, transparency, and traceability of AI decisions. All actions are logged, and developers retain final authority over system outputs.
The pipeline is designed in alignment with the EU AI Act, particularly emphasizing human oversight, accountability, and risk mitigation through controlled validation and audit mechanisms.

7. Continuous Learning & Feedback Loop

Approved fixes and developer feedback are stored back into the knowledge base.
The system updates:

- Category summaries
- Retrieval relevance
- Confidence scoring models
- File-location mappings
-

This creates a closed-loop improvement cycle, enabling BugScout AI to become more accurate, faster, and context-aware with every validated fix.

Models and Tools

LLM

Primary Model: OpenAI GPT-4o-mini. Used for all four agents: Issue Monitoring Agent, Solution Agent, Self-Learning Agent, and Codebase Crawler Agent. Chosen for cost-effectiveness, speed, and strong reasoning capabilities. JSON mode enabled for structured outputs. Max tokens: 2000 (Issue Monitoring), 2500 (Solution Agent), 3000 (Self-Learning Agent), 2000 (Codebase Crawler Agent).

Embeddings

Model: OpenAI text-embedding-3-small (1536-dimensional embeddings). Batch processing: 100 documents per batch for efficiency. Used for all vectorization: monitoring data, issues, logs, PostHog events.

Retrieval

Vector Database: ChromaDB Cloud. Collections: monitoring, issues, logs, posthog_events. Semantic similarity search using cosine distance. Metadata filtering by severity, table type, recording ID. Automatic embedding generation via custom OpenAI embedding function.

Reranking

Hybrid Approach: Recency-based ranking (most recent logs filtered by category prioritized, 25 entries), developer rating weighting (high-rated solutions 4-5 boost agent confidence scores), semantic similarity (vector search finds contextually similar issues even without exact matches), codebase map alignment (file path matching improves code location accuracy).

Hosting/UI

Frontend: Next.js 14 with React, TypeScript, Tailwind CSS.
Backend: Next.js API Routes (serverless functions)
Database: Neon PostgreSQL (serverless Postgres); Vector Store: ChromaDB Cloud.
Authentication: Clerk. Analytics: PostHog (session replays, events, feature flags)
Deployment: Vercel with cron jobs for auto-sync.

Why this works?

1. **Multi-Agent Architecture**

Four specialized agents working in coordination: Issue Monitoring Agent (detection and classification), Solution Agent (fix generation with context from other agents), Self-Learning Agent (knowledge summarization by category), Codebase Crawler Agent (dynamic codebase analysis). Each agent receives specialized context, reducing cognitive load and improving accuracy. Allows independent optimization of each stage. Self-Learning Agent enables faster context retrieval through category-based summarization.

2. **Real-Time Data Processing**

Direct integration with PostHog provides live session data. Issues are detected as they occur, not in retrospect. Enables proactive issue resolution before widespread user impact.

3. **Self-Learning Knowledge Base**

Developer ratings create a feedback loop that improves over time. Self-Learning Agent processes all approved fixes and creates category-specific summaries. Category summaries enable faster context retrieval for the Solution Agent. Vector similarity search enables retrieval of relevant past solutions even with slight variations. High-rated solutions inform confidence scoring, making the system more reliable. Summarization reduces token usage while preserving critical knowledge patterns.

4. **Codebase-Aware Context**

CODEBASE_MAP.json provides static file structure and component roles for smaller codebases. Codebase Crawler Agent dynamically analyzes large codebases (1000+ files) in real-time. Agents can pinpoint exact code locations and suggest file-specific fixes. Reduces hallucination and improves actionable suggestions. Adapts to codebase changes without manual map updates.

5. **Structured Data Pipeline**

NeonDB ensures data integrity and ACID compliance. ChromaDB enables fast semantic search at scale. Automatic synchronization keeps both stores in sync. Deduplication prevents redundant processing.

6. **Multi-Signal Analysis**

Combines multiple signals: exceptions, rage clicks, dead clicks, console errors, page events. Pattern recognition across sessions identifies recurring issues. Severity classification considers impact, not just error presence.

Demonstrated Improvement

Baselines

Baseline 1: Manual issue detection (human review of PostHog session replays)
Baseline 2: Rule-based detection (threshold-based alerts for error counts, rage clicks)
Baseline 3: Generic LLM without context (GPT-4o-mini with only session data, no codebase map or past solutions)

Method

Test Dataset: 50 real session recordings from production PostHog data
Evaluation Metrics: Issue Detection (rate of actual issues correctly identified), False Positive Rate (rate of non-issues flagged as issues), Code Location Accuracy (% of issues with correct file path identification), Fix Quality Score (developer rating 1-5 for suggested fixes), Time to Resolution (time from issue detection to fix suggestion), Confidence Score Accuracy (correlation between agent confidence and developer rating)

Metric

Primary: Developer Rating (1-5 scale for suggested fixes)
Secondary: Issue Detection Precision/Recall, Code Location Accuracy, Confidence Score Correlation

Test Prompts

Test Prompt 1: Exception Detection
Session ID: rec_abc123, Console errors: 3, Exception: "Cannot read property 'value' of undefined", URL: /checkout/payment, Element: payment-form input[name='cardNumber'], Timestamp: 2025-02-10T14:23:15Z

Test Prompt 2: UX Friction - Rage Clicks
Session ID: rec_def456, Rage clicks: 5, Dead clicks: 2, URL: /dashboard/settings, Element: button.submit-btn, Duration: 45s

Test Prompt 3: Pattern Recognition
Multiple sessions with same error: rec_ghi789 (Network timeout at /api/data), rec_jkl012 (Network timeout at /api/data), rec_mno345 (Network timeout at /api/data)

Results

Baseline: Manual Detection: 85% detection rate, 2-4 hours per issue, 0% automation. Rule-Based: 60% detection rate, high false positives (40%), no fix suggestions. Generic LLM: 70% detection rate, 45% code location accuracy, average developer rating 2.8/5.

Enhanced system

Issue Detection Rate: 92% vs 70% baseline
False Positive Rate: 8% vs 40% baseline
Code Location Accuracy: 87% vs 45% baseline
Average Developer Rating: 4.2/5 vs 2.8/5 baseline
Time to Resolution: 30 seconds vs 2-4 hours manual
Confidence Score Correlation: 0.78 (strong positive correlation with developer ratings)

Lift

Detection Rate: 31% improvement over generic LLM
Code Location Accuracy: 93% improvement (87% vs 45%)
Developer Satisfaction: 50% improvement (4.2 vs 2.8)
Automation: 100% automated vs 0% baseline
Time Savings: 99% reduction in time to resolution

Example Measured Improvement

Prompt

Session ID: rec_xyz789, Console errors: 2, Exception: TypeError: Cannot read property 'map' of null, URL: /products/list, Element: div.product-grid, Timestamp: 2025-02-10T15:30:22Z, Page events: 12 pageviews, 3 clicks on product cards

Baseline outcome

Issue Detected: Yes (generic JavaScript error). Category: Incorrect (classified as performance instead of errors). Code Location: Incorrect (app/page.tsx instead of components/ProductGrid.tsx). Fix Suggestion: Generic - "Add null check" without specific code. Developer Rating: 2/5 (not actionable)

title text	description text	severit...	suggested_fix text	agent_confidence_scor...
Login button unrespons…	User clicked Login multiple …	High	Disable button on subm…	0.46
Slow response on filte…	User changed filter and clic…	Low	Optimize filter query;…	0.15
Table row click and bu…	Clicking row button triggere…	Medium	Stop propagation on bu…	0.46
Logout redirects to wr…	After logout user landed on …	Low	Set redirect URL in au…	0.48
Dead click on Submit b…	Submit button clicked with n…	Medium	Check form onSubmit ha…	0.36
Form clears on validat…	Single field error caused en…	High	Preserve form values o…	0.36
Back button loses form…	Browser back cleared the for…	High	Use sessionStorage or …	0.32
Slow initial load on d…	Dashboard took several secon…	High	Lazy load below-fold; …	0.40
Duplicate network requ…	Refocusing tab triggered dup…	Medium	Avoid refetch on focus…	0.42
Switch toggle ignores …	First click on toggle did no…	Medium	Check controlled vs un…	0.44
Dark mode toggle not p…	After refresh, theme reverte…	Low	Persist preference in …	0.48
Notification badge not…	After viewing notifications,…	Low	Mark as read and refet…	0.38
Navigation link not re…	Sidebar link clicked multipl…	Medium	Verify Link href and p…	0.40
Time zone wrong in rep…	Report timestamps showed wro…	Medium	Store UTC; display in …	0.28
Pagination resets to p…	After changing filters, pagi…	Low	Reset page index when …	0.32
Avatar upload crop not…	After cropping, saved image …	Low	Apply crop before uplo…	0.38
File upload progress s…	Progress bar stopped at same…	Medium	Handle xhr progress an…	0.38
Modal fails to open on…	User clicked Add Guest but m…	Medium	Debounce modal trigger…	0.28
Search returns no resu…	First search empty; second s…	Medium	Ensure query is sent a…	0.28
Chart not updating wit…	New data loaded but chart st…	Medium	Key chart by data vers…	0.46
Rage clicks on Refresh	User clicked Refresh multipl…	High	Disable Refresh while …	0.40
Autocomplete suggestio…	Typing in one field showed s…	Medium	Scope suggestion sourc…	0.34
Slow response on filte…	User changed filter and clic…	Low	Optimize filter query;…	0.42
Console error on route…	Navigating away threw React …	Medium	Cancel async or ignore…	0.40

Enhanced Outcome

Issue Detected: Yes (correctly identified as js-frontend-errors). Category: Correct (errors - js-frontend-errors). Code Location: Correct (components/ProductGrid.tsx). Fix Suggestion: 1. Add null check before mapping: products?.map(...) or products || [].map(...); 2. Initialize products state as empty array: const [products, setProducts] = useState([]); 3. Add loading state to prevent rendering before data fetch completes. Code Edits: components/ProductGrid.tsx - const items = products.map(...) → const items = (products || []).map(...). Agent Confidence Score: 0.85 (high, due to similar past solution with rating 5/5). Developer Rating: 5/5 (actionable, specific, correct location)

id uuid	title text	description text	severit...	agent_confidence_scor...
12a5ee87-b569-4165-9ac…	High User Frustration …	The user exhibited 5 instanc…	High	0.90
14f4c0b6-5cde-4263-912…	User Frustration with …	This session recorded 6 rage…	High	0.80
1ead1825-ba69-4919-b0d…	User Frustration with …	During the session, the user…	Medium	1.00
2b270d6a-d09f-4866-8e7…	High User Frustration …	The user demonstrated signif…	High	1.00
3f3c87d9-a4b3-4dd5-9a2…	User Frustration with …	The user exhibited 5 instanc…	High	0.90
4cbc2597-4221-411b-9c6…	User Frustration with …	The user exhibited 5 instanc…	High	1.00
5a90a90a-4af2-467b-aeb…	Repeated User Rage Cli…	In this session, the user de…	High	0.90
5eb2aedc-f137-46a8-b17…	High Rage Clicks on Da…	The user session displayed 5…	High	0.80
5f0b1611-174a-45c5-86c…	Multiple JavaScript Ex…	During this session, the use…	High	0.90
62a05b33-5509-46ab-a3a…	High User Frustration …	User displayed repeated frus…	High	0.80
66eeeb6f-16f8-40bb-91a…	User Frustration with …	User displayed frustration w…	Medium	1.00
67b1c8fa-1e58-432f-902…	User Frustration with …	This session recorded 6 rage…	High	1.00
725ed3ff-25ac-4897-a9d…	Rage Click and Dead Cl…	The session recorded a rage …	Medium	0.90
758f5055-0efe-48d0-83d…	Single Rage Click on D…	The user experienced a singl…	Medium	0.90
8203b571-19b3-452b-93a…	Rage and Dead Clicks I…	The recording shows a total …	High	0.80
87bb648c-63f0-4977-a0b…	Console Errors Affecti…	This session has recorded 4 …	High	0.90
a977a68e-56b5-41e2-aef…	User Frustration Due t…	During the session, the user…	Medium	0.85
b7927fab-9ae3-4569-b3f…	Multiple JavaScript Ex…	The session recorded 3 JavaS…	High	1.00
e0b903eb-ea82-4f62-ba8…	High User Frustration …	In this session, the user de…	High	1.00
eff92829-1f38-4dc4-9c4…	Minor User Frustration…	The session featured 1 insta…	Low	0.75
fa0c2473-8f7d-4a7d-9b2…	Multiple JavaScript Ex…	The session recorded 3 JavaS…	High	0.80
ff0fdf20-084d-4308-ac4…	Rage Clicks and Dead C…	The user demonstrated frustr…	High	0.90

Evidence Used

Past Solution Retrieved: Similar issue "Cannot read property 'map' of null" with rating 5/5. Codebase Map: Identified components/ProductGrid.tsx from URL /products/list and element div.product-grid. PostHog Taxonomy: Correctly mapped to js-frontend-errors category. Vector Similarity: Retrieved log entry with 0.82 cosine similarity score.

Appendix Links

YouTube Demo Link: https://youtu.be/mejBsvvPU5A

Live Hosted Website Link: https://bugscoutai.vercel.app/